

vSphere Export Guide

This guide covers h2kvm's direct export capabilities for VMware vSphere environments.

Table of Contents

- [Overview](#)
- [Export Modes](#)
- [ExportOptions API](#)
- [Export Workflows](#)
- [Authentication](#)
- [Best Practices](#)
- [Troubleshooting](#)

Overview

h2kvm provides multiple methods to export VMs from vSphere environments:

1. **Direct Export** - Convert VMware disks directly to QCOW2/RAW using internal converters
2. **HTTP Download** - Download VM folders via vSphere's HTTP API
3. **OVF/OVA Export** - Export as standard OVF or OVA packages over an HTTP NFC lease

All export modes support: - Snapshot management (create, use existing, no snapshot) - Credential resolution (environment, config file, interactive) - Progress tracking and logging - Error recovery and retry logic - **VM hardware info extraction** — memory (MiB), vCPUs, NIC count, and total disk size are extracted from `govc vm.info -json` and propagated to the domain emitter for accurate libvirt XML generation

Export Modes

1. Direct Export (`export_mode="export"`)

Converts VMware disks directly to KVM-compatible formats using h2kvm's internal converters.

Features: - Guest OS detection and modification - Filesystem fixes (fstab, network config) - Initramfs regeneration - Driver injection

Use Case: Full V2V migration with guest modifications

Example:

```

from h2kvm.vmware.clients.client import VMwareClient, ExportOptions

client = VMwareClient(
    vcenter_host='vcenter.example.com',
    username='administrator@vsphere.local',
    password='password'
)

await client.async_export_vm(
    vm_name='production-web',
    output_dir='/data/exports',
    export_mode='export' # Direct export with conversion
)

```

2. HTTP Download (export_mode="download_only")

Downloads exact VM folder contents from the datastore using vSphere's /folder HTTP API.

Features: - Preserves original VMware formats - Downloads all VM files (VMDK, VMX, etc.) - Fast transfer via HTTPS - No conversion overhead

Use Case: Backup, archival, or manual conversion later

Example:

```

options = ExportOptions(
    vm_name='backup-vm',
    output_dir='/backups',
    export_mode='download_only' # HTTP download, no conversion
)

await client.export_vm(options)

```

3. OVF Export (export_mode="ovf_export")

Exports VM as standard OVF format using VMware APIs.

Use Case: Portable VM format for import to other platforms

4. OVA Export (export_mode="ova_export")

Exports VM as single OVA archive file.

Use Case: Simplified distribution and portability

ExportOptions API

The `ExportOptions` class configures export behavior:

```

from h2kvm.vmware.clients.client import ExportOptions

options = ExportOptions(
    # Required
    vm_name='target-vm',
    output_dir='/exports',

    # Export mode
    export_mode='export', # 'export', 'download_only', 'ovf_export', 'ova_export'

    # Snapshot handling
    snapshot_mode='create', # 'create', 'use_existing', 'none'
    snapshot_name='migration-snapshot',

    # Conversion options (for export mode)
    output_format='qcow2', # 'qcow2', 'raw'
    compress=True,

    # Authentication
    vcenter_host='vcenter.example.com',
    username='admin@vsphere.local',
    password_file='/secure/vcpass.txt',

    # Performance
    parallel_downloads=True,
    max_workers=4,

    # Advanced
    extra_args=['--verbose'],
    timeout=3600,
)

```

Key Parameters

Export Mode

- `export` - Full conversion with guest modifications
- `download_only` - Raw file download via HTTP
- `ovf_export` - Standard OVF format
- `ova_export` - Single OVA archive

Snapshot Mode

- `create` - Create new snapshot for export
- `use_existing` - Use existing snapshot by name
- `none` - Export from current state (downtime required)

Output Format

- `qcow2` - QEMU Copy-On-Write (recommended)
- `raw` - Raw disk image
- `vdi` - VirtualBox format

Export Workflows

Basic Export Workflow

```
import asyncio
from h2kvm.vmware.clients.client import VMwareClient

async def export_vm():
    client = VMwareClient(
        vcenter_host='vcenter.example.com',
        username='admin@vsphere.local',
        password='password'
    )

    await client.async_export_vm(
        vm_name='web-server-01',
        output_dir='/data/migrations',
        export_mode='export'
    )

asyncio.run(export_vm())
```

Batch Export with Progress

```
from h2kvm.manifest.batch_progress import ProgressTracker

async def batch_export(vm_list):
    client = VMwareClient(...)
    tracker = ProgressTracker('/tmp/progress.json', 'batch-export', len(vm_list))

    for vm_name in vm_list:
        tracker.start_vm(vm_name)
        tracker.update_vm_stage(vm_name, 'export')

        try:
            await client.async_export_vm(
                vm_name=vm_name,
                output_dir=f'/exports/{vm_name}',
                export_mode='export'
            )
            tracker.complete_vm(vm_name, success=True)
        except Exception as e:
            tracker.complete_vm(vm_name, success=False, error=str(e))

    tracker.cleanup()
```

Export with Snapshot Management

```
# Create snapshot for export
options = ExportOptions(
    vm_name='production-db',
    output_dir='/backups',
    export_mode='download_only',
    snapshot_mode='create',
    snapshot_name='pre-migration-backup',
    snapshot_description='Snapshot for migration testing',
    snapshot_memory=False # Faster, no RAM state
)

await client.export_vm(options)
```

Authentication

Environment Variables

```
export VSPHERE_HOST='vcenter.example.com'
export VSPHERE_USER='admin@vsphere.local'
export VSPHERE_PASSWORD='password'

h2kvmctl ...
```

Password File

```
options = ExportOptions(
    vcenter_host='vcenter.example.com',
    username='admin@vsphere.local',
    password_file='/secure/.vcpass', # One line: password
    ...
)
```

Config File

```
# config.yaml
cmd: vsphere
vs_action: export
vcenter_host: vcenter.example.com
vcenter_user: admin@vsphere.local
vc_password_env: VC_PASSWORD

export_mode: export
output_dir: /data/exports
```

```
h2kvm --config config.yaml --vm-name web-server-01
```

Best Practices

1. Use Snapshots for Live VMs

Always create snapshots when exporting running VMs:

```
export_mode='export',
snapshot_mode='create',
snapshot_name=f'migration-{timestamp}',
snapshot_memory=False # Faster
```

2. Plan Bandwidth Usage

For large VMs, schedule exports during off-peak hours:

```
# Limit parallel downloads
max_workers=2 # Instead of default 4

# Use compression
compress=True
```

3. Validate Exports

Always validate exported disks:

```
from h2kvm.validation import DiskValidator

validator = DiskValidator()
report = validator.validate({
    'output_path': '/exports/vm-disk.qcow2',
    'format': 'qcow2',
    'minimum_size': 10 * 1024 * 1024 * 1024 # 10 GB
})

if report.has_errors():
    print("Validation failed!")
```

4. Clean Up Snapshots

Remove temporary snapshots after export:

```
await client.async_delete_snapshot(
    vm_name='production-db',
    snapshot_name='migration-snapshot'
)
```

5. Monitor Progress

Use progress tracking for visibility:

```
from h2kvm.manifest.batch_progress import ProgressTracker

tracker = ProgressTracker(
    progress_file='/tmp/export-progress.json',
    batch_id='nightly-backup',
    total_vms=50
)
```

Troubleshooting

Connection Issues

Error: Connection refused or TLS handshake failed

Solution:

```
# Verify connectivity
curl -k https://vcenter.example.com/

# Check certificate
export VSPHERE_VERIFY_SSL=false # For testing only

# Check credentials
export VSPHERE_DEBUG=1
```

Slow Export Performance

Issue: Export taking too long

Solutions: 1. Use `export_mode='download_only'` for raw files 2. Increase `max_workers` for parallel downloads 3. Check network bandwidth between h2kvm host and vSphere

Snapshot Failures

Error: Snapshot creation failed

Solutions: 1. Verify VM has no existing snapshot locks 2. Check datastore has sufficient space 3. Use `snapshot_memory=False` to avoid RAM dump 4. Try `snapshot_mode='none'` with VM powered off

Permission Denied

Error: Permission denied during export

Solution: 1. Verify user has VM export permissions in vSphere 2. Check datastore accessibility 3. Ensure snapshot creation rights 4. Review vCenter role assignments

See Also

- [vSphere Design](#) - Architecture details
- [Batch Migration Guide](#) - Batch processing
- [Library API](#) - Programmatic usage
- [Troubleshooting](#) - Common issues

Examples

Complete examples are available in: - `examples/yaml/` - YAML configuration samples - `examples/json/` - JSON manifest examples - `test-confs/` - Test configurations

For more information, see the main [documentation index](#).